



DairyDesk

Developer Guide — rules, conventions and pre-flight checks

Audience Anyone adding features or changing existing code

Stack Django 5.1 + DRF · React 19 + Vite 8 · PostgreSQL 16

Runs on Docker (Python 3.13 / Node 22 pinned by the images)

Read first `README.md` for setup, `PROJECT_SPEC.md` for the spec

1. Before you write any code

DairyDesk is a small codebase with a few sharp edges. Almost every bug introduced here comes from missing one of the rules in section 3 — they are not style preferences, they are correctness requirements around **money**, **stock** and **who is allowed to see what**.

Get the app running

```
git clone https://github.com/TanmayKamble004/DairyDesk.git
cd DairyDesk
docker compose up --build
```

That is the entire setup — no virtualenv, no `npm install`, no `.env` to copy. Frontend on `:5173`, API on `:8000`, admin at `/admin`. Log in as `owner / owner123` or `staff / staff123`.

YOU DO NOT NEED PYTHON OR NODE INSTALLED

The containers pin Python 3.13 and Node 22. If you have a local venv from before, ignore it — do not run the servers on the host and in Docker at the same time, they will fight over ports 8000 and 5173.

Understand the domain first

Four models carry all the business logic. Skim them before touching anything:

Model	What it means — and the part people get wrong
Product	A sellable item with a <i>current</i> <code>selling_price</code> . Changing this price must never change past orders.
StockBatch	A physical delivery with a quantity and an expiry date. Stock lives in batches, never on the product.
Order / OrderItem	An order deducts stock at creation time and <i>snapshots</i> the price into <code>OrderItem.unit_price</code> .
Invoice	Created automatically when an order becomes <code>delivered</code> . Owner-only data.

2. Pre-flight checklist

Run through this before opening a pull request. It takes two minutes and catches nearly everything.

Any change

- ☐ The app still starts clean: `docker compose down && docker compose up` with no errors in the logs.
- ☐ You logged in as **both** `owner` and `staff` and clicked through the affected pages.
- ☐ No secrets, API keys, or absolute URLs hardcoded — config comes from environment variables only.
- ☐ No `console.log` / `print()` debugging left behind.
- ☐ `git status` is clean of junk — no `.env`, no `__pycache__`, no `node_modules`.

If you touched a model

- ☐ Generated the migration: `docker compose exec backend python manage.py makemigrations`

- ☐ **Committed the migration file.** Forgetting it breaks every teammate's database.
- ☐ Confirmed it applies: `docker compose restart backend` shows `Applying ... OK`.
- ☐ Checked nothing is pending: `makemigrations --check --dry-run` prints *No changes detected*.
- ☐ Updated `seed_demo.py` if the new field is needed for a realistic demo.

If you added or changed an API endpoint

- ☐ Route registered in `backend/core/urls.py` (router for viewsets, `path()` for APIViews).
- ☐ New fields added to the serializer's explicit `fields` list — they will not appear otherwise.
- ☐ Decided the permission deliberately. The global default is `IsAuthenticated`; financial data needs `IsOwner`.
- ☐ Tested it while logged in as `staff`, not just `owner`.

If you touched the frontend

- ☐ Used the shared `api` client, not raw `axios` or `fetch`.
- ☐ Reused components from `components/ui.jsx` instead of restyling from scratch.
- ☐ Errors surfaced through `apiErrorMessage(error)`.
- ☐ Owner-only pages wrapped in `<RequireOwner>` **and** protected server-side.

3. The rules that actually matter

These encode business correctness. Breaking one produces wrong money, wrong stock, or a data leak — usually with no error message.

3.1 Role gating lives in two places. Always do both.

Financial data is owner-only. That is enforced on the server by `IsOwner` (`core/permissions.py`) and in the UI by `RequireOwner` (`App.jsx`).

Why: the frontend guard is cosmetic. A staff user who calls the API directly gets the data anyway unless the server refuses. Conversely, server-only protection gives staff a broken page instead of a hidden one. **Both, every time.**

3.2 Dashboard KPIs are filtered by role in the view

`DashboardView` only puts `total_available_stock_value`, `todays_sales_total` and `unpaid_invoice_count` in the response when `is_owner`. The frontend renders whichever keys are present.

Why: adding a money KPI outside the `if is_owner` block silently leaks revenue data to staff, and nothing in the UI will look wrong.

3.3 `unit_price` is a historical snapshot — never recompute it

`OrderItem.unit_price` is copied from the product at order-creation time. Totals must always be derived from it, never from `product.selling_price`.

Why: if you read the live price, raising a product's price retroactively rewrites every historical invoice. Use `item.line_total`.

3.4 Stock deduction is FIFO and must run inside a transaction

`deduct_stock_fifo()` in `core/services.py` takes from the oldest non-expired batches first and uses `select_for_update()`. Every caller must wrap it in `transaction.atomic()` — see `OrderSerializer.create`.

Why: the row locks only hold inside a transaction. Outside one, two simultaneous orders can sell the same batch twice. It also validates all-or-nothing: if any product is short, it raises and deducts nothing.

3.5 Order status moves one way only

`pending → processed → delivered`, enforced by `ORDER_STATUS_TRANSITIONS`. `status` is read-only on `OrderSerializer`; changes go through `PATCH /api/orders/{id}/` only.

Why: reaching `delivered` creates an invoice. Allowing backward moves would orphan invoices and corrupt the sales figures.

3.6 Delivering an order auto-creates its invoice

`ensure_invoice(order)` is idempotent (`get_or_create`). Never create `Invoice` objects by hand elsewhere.

Why: one order has exactly one invoice (`OneToOneField`), and the total must be computed the same way everywhere.

3.7 `expiry_status` is computed, never stored

It is a `@property` on `StockBatch` derived from today's date and `AGEING_THRESHOLD_DAYS = 3`. Fresh > 3 days left, ageing ≤ 3 days, expired past.

Why: a stored column would be stale the next morning. This one constant drives the 3D shelf colours, the dashboard warning counts and the inventory endpoint — change it and all three move together.

3.8 Expired stock counts as visible, not as available

Across `InventoryView`, `DashboardView` and `Product.available_quantity`: expired batches are shown and counted, but excluded from available quantity and stock value. Batches with `quantity == 0` are skipped entirely.

Why: the shop still physically holds expired stock and needs to see it, but must never sell it or count it as money.

3.9 Use the custom user model

`AUTH_USER_MODEL = "core.User"`, with a `role` field of `owner` / `staff`. Reference it via `get_user_model()` or `settings.AUTH_USER_MODEL`.

Why: importing `django.contrib.auth.models.User` silently targets the wrong table.

3.10 Deletions are deliberately blocked

`Customer → Order` and `Product → OrderItem` use `on_delete=PROTECT`.

Why: deleting a customer or product that appears in order history would destroy the audit trail. If a "delete" feature is requested, add a deactivation flag instead of weakening these.

4. Backend conventions

Where	Convention
<code>models.py</code>	Business <i>data</i> . Derived values are <code>@property</code> , not columns.
<code>services.py</code>	Business <i>rules</i> shared by more than one caller (stock deduction, invoice creation). Put logic here, not in views.
<code>serializers.py</code>	Validation and write logic. Multi-step writes go in <code>create()</code> / <code>update()</code> wrapped in <code>transaction.atomic()</code> .
<code>views.py</code>	Thin. Wire up querysets, permissions and serializers — delegate real work to services.
<code>permissions.py</code>	Reusable permission classes. Add here rather than checking roles inline in views.

Things to keep doing

- **Explicit serializer `fields` lists.** Never `fields = "__all__"` — it exposes new columns the moment they are added.
- `select_related` / `prefetch_related` **on every list queryset.** The existing viewsets all do this; copy the pattern or you will add N+1 queries.
- **Money is `DecimalField`, never `float`.** Serialize totals with `str()`, as the dashboard does.
- **Dates via `timezone.localtime()`, not `date.today()`** — `USE_TZ` is on.
- **Validation errors as DRF `ValidationError`** with a field key, so the frontend's `apiErrorMessage` can render them.

5. Frontend conventions

- **Always import `api` from `src/api/client.js`.** It attaches the JWT and transparently refreshes it on a 401 (single-flight, so parallel calls refresh once). A raw `fetch` or bare `axios` call bypasses both and will 401 randomly after eight hours.
- **Reuse `components/ui.jsx`** — `Card`, `PageHeader`, `Spinner`, `ErrorAlert`, `Badge`, `Th`, `Td`, plus the `inputClass` / `buttonPrimary` / `buttonSecondary` class strings.
- **Every page handles three states:** loading, error, loaded. Copy the pattern from `pages/Inventory.jsx`.
- **Format currency with the existing `formatINR` helper** (`Dashboard.jsx`) rather than inventing another one.
- **Tailwind utility classes only.** No new CSS files — Tailwind 4 is wired in through the Vite plugin.
- **Never read tokens from `localStorage` directly.** Use `getAuth()` / `useAuth()`.

THE 3D SHELF IS PERFORMANCE-SENSITIVE

`InventoryShelf.jsx` renders with react-three-fiber. Objects created inside the render path (geometries, materials, arrays) get rebuilt every frame and will tank the frame rate. Memoise them, and test with the full seeded dataset rather than two products.

6. Working with Docker day to day

Your files are mounted into the containers, so **editing** `.py`, `.jsx` or `.css` **needs no command** — Django reloads in about a second, Vite hot-swaps the module in the browser. Only these need action:

Situation	Command
Added a Python package	<code>docker compose up -d --build backend</code>
Added an npm package	<code>docker compose up -d --build --renew-anon-volumes frontend</code>
Changed a model	<code>docker compose exec backend python manage.py makemigrations</code>
Pulled new migrations	<code>docker compose restart backend</code>
Any <code>manage.py</code> command	<code>docker compose exec backend python manage.py <cmd></code>
Reading logs	<code>docker compose logs -f backend</code>
Shell in a container	<code>docker compose exec backend bash</code> (frontend is Alpine → use <code>sh</code>)

THREE COMMANDS THAT DESTROY DATA

`docker compose down -v` — the `-v` deletes the Postgres volume and everything in it. Plain `down` is the safe one.

`python manage.py seed_demo` — **clears** products, batches, customers, orders and invoices before reseeding. It is not additive.

`python manage.py flush` — wipes all rows. Rarely what you want here.

SAFE BY DESIGN

Demo data is seeded **only into an empty database**. Restarting containers, rebuilding images and running `docker compose down` all preserve your data.

Do not change these without understanding why

- `DB_HOST: db` and `DB_PORT: 5432` are **hardcoded in** `docker-compose.yml`, not read from `.env`. Inside the Docker network the backend reaches Postgres by service name on its container port; the `.env` values describe how your *host* reaches it and may differ (5433 is common when 5432 is taken).
- `.gitattributes` **forces LF endings on** `*.sh`. If `docker-entrypoint.sh` ever gets CRLF, the container dies with a baffling `no such file or directory`.
- `node_modules` **lives in a Docker volume**, deliberately masking the host copy (which may hold Windows binaries). That is why npm changes need `--renew-anon-volumes`.

7. Testing — read this honestly

THERE IS NO AUTOMATED TEST SUITE

`backend/core/tests.py` is still the empty Django stub, and there are no frontend tests. Nothing will catch a regression for you. Until that changes, **manual verification is mandatory**, and the burden is on the person making the change.

Minimum manual pass for a backend change

1. Log in as `owner` — dashboard, inventory, orders and invoices all load.
2. Log in as `staff` — financial KPIs are absent and `/invoices` is not reachable.
3. Create an order — stock drops from the oldest batch first.
4. Try to order more than is available — you get a clear error and **nothing** is deducted.
5. Move an order `pending → processed → delivered` — an invoice appears with the correct total.
6. Re-check the dashboard — today's order count and sales total moved as expected.

Reset to a known state whenever you need one:

```
docker compose exec backend python manage.py seed_demo
```

The seed deliberately includes fresh, ageing *and* expired batches so all three shelf colours are visible — keep it that way if you edit it.

ADDING TESTS IS WELCOME

The highest-value first tests are `deduct_stock_fifo` (ordering, shortage roll-back, concurrency), the order status transition rules, and a staff-cannot-read-invoices permission test. Run them with

```
docker compose exec backend python manage.py test .
```

8. Git conventions

- Branch off `main`; do not commit directly to it.
- Existing history uses short prefixed subjects — `docs: ...`, `Phase 4: ...`. Keep messages in the imperative and scoped to one change.
- **Never commit** `.env`, `.venv/`, `node_modules/`, `__pycache__/`, or `*.sqlite3`. `.gitignore` covers these — do not force-add them.
- **Always commit** migration files alongside the model change that produced them, and `package-lock.json` alongside a dependency change.
- Pull, then `docker compose restart backend` before assuming something is broken — it is often just an unapplied migration.

9. Common mistakes

Symptom	Real cause
Field added but missing from the API response	Not in the serializer's explicit <code>fields</code> list.
<code>relation "core_..." does not exist</code>	Migration not generated, or generated but not committed.
Teammate's app broke after your merge	Model change committed without its migration file.
Staff can see money	Missing <code>IsOwner</code> server-side, or a KPI added outside the <code>is_owner</code> block.
Requests 401 after a few hours	Bypassed the shared <code>api</code> client, so the refresh interceptor never ran.
Old invoice totals changed by themselves	Reading <code>product.selling_price</code> instead of <code>OrderItem.unit_price</code> .
Stock went negative or double-sold	<code>deduct_stock_fifo</code> called outside <code>transaction.atomic()</code> .
<code>Cannot find module</code> after pulling	New npm package — rebuild with <code>--renew-anon-volumes</code> .
Port 8000 or 5173 already in use	A host-run <code>runserver</code> / <code>npm run dev</code> is still going, or set <code>BACKEND_PORT</code> / <code>FRONTEND_PORT</code> .
All your data vanished	<code>docker compose down -v</code> . The volume is gone; reseed.

DairyDesk Developer Guide · keep this in sync with `README.md` and `PROJECT_SPEC.md` · source:
`docs/developer-guide.html`